

***Multi – level linked list***  
***–Delete At Position(Main List)***  
***–Time Complexity***

***Program: –***

```
struct Node
{
    int data;
    struct Node *next; // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
};

Node *head = nullptr ; // Global Head Pointer
// Helper: Create a new node with malloc
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}

void deleteAtPosition()
{
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;

    if (head == nullptr)
        return;
```

```

Node *temp = head;

if (pos == 1)
{
    head = head->next;
    free(temp);
    return;
}

for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}

if (temp == nullptr || temp->next == nullptr)
{
    cout << "Position out of range." << endl;
    return;
}

Node *toDelete = temp->next;
temp->next = toDelete->next;
free(toDelete);
cout << "Deleted at position " << pos << endl;
}
...
int main()
{
    int choice;
    while (true)
    {
        cout << "7. Delete at Position (Main)" << endl;
        cin >> choice;

        switch (choice)
        {
            case 7:
                deleteAtPosition();

```

```

        break;
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}

```

### **Time Complexity Analysis**

**1. Absolute Best Case: When List is Empty.**

*When List is not Empty, but we delete first node (pos = 1), also our best case but not Absolute one.*

**2. Worst case or Absolute worst case when Temp = NULL, the loop's inner statement runs `n` times i.e. POS or `position` not found, hence :  $O(n)$ .**

*When we want to delete 1st node, loop runs upto 0 time.*

*When we want to delete 2nd node, loop runs upto 0 time.*

*When we want to delete 3rd node, loop runs upto 1 time.*

.

.

*When `n - 1` node is to be deleted :*

*Upper Bound : pos - 2, when pos = n - 1, then :*

*loop runs:  $(n - 1 - 2) = n - 3$  times.*

.  
*When `n` node is to be deleted :*

*Upper Bound : pos - 2, when pos = n, then :*

*loop runs: (n - 2) = n - 2 times.*

### *Average Case*

*Deleted position from 2 to N, for loop runs from 1 to N - 2.*

*And when the position is uniformly distributed from 1 to N:*

*(Average Case i. e. Undetermined Position)*

*When we want to delete 1st node , loop runs upto 0 time.*

*When we want to delete 2nd node , loop runs upto 0 time.*

*When we want to delete 3rd node , loop runs upto 1 time.*

.

.

*When `n - 1` node is to be deleted :*

*Upper Bound : pos - 2, when pos = n - 1, then :*

*loop runs: (n - 1 - 2) = n - 3 times.*

*When `n` node is to be deleted :*

*Upper Bound : pos - 2, when pos = n, then :*

*loop runs: (n - 2) = n - 2 times.*

*When we want to delete  $\sum_{i=1}^n i$  nodes , loop runs  $\sum_{i=1}^{n-2} i$*

Now, as we take all the possibilities then, we take the best case also i.e.  $O(1)$ , when position = 1. Considering  $T(P)$  is time complexity of position  $P$ .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ 1 & , \text{when } P = 2. \\ P - 2 & , \text{when } P = 3, 4, \dots, N. \end{cases}$$

Now, If there is  $N$  no. of nodes there we can delete data at  $N$  positions.

For position 1, the probability of deletion is  $\frac{1}{N}$ .

For position 2, the probability of deletion is  $\frac{1}{N}$ .

.

For position  $N$ , the probability of deletion is  $\frac{1}{N}$ .

Hence probability of deletion will always remain  $\frac{1}{N}$ .

$\therefore$  Average Time Complexity is : probability of deletion  $\times$  (number of possible iterations + constant time complexities for left over positions) .

$$= \left( \frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 2 \right) + \frac{1}{N} \times 1 + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 2) + 1 + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{n-2} i + 2 \right\}$$

$$= \frac{1}{N} \left\{ \frac{(n-2)(n-1)}{2} + 2 \right\}$$

$$= \frac{1}{N} \left\{ \frac{n^2 - 3n + 6}{2} + 2 \right\}$$

$$= \left\{ \frac{n^2 - 3n + 6}{2n} + \frac{2}{n} \right\}$$

$$= \left\{ \frac{n^2}{2n} - \frac{3n}{2n} + \frac{6}{2n} + \frac{2}{n} \right\}$$

$$= \left\{ \frac{n}{2} - \frac{3}{2} + \frac{3}{n} + \frac{2}{n} \right\}$$

**Applying Big Theta in the equation:**

$$= \Theta\left(\frac{n}{2}\right) - \Theta\left(\frac{3}{2}\right) + \Theta\left(\frac{3}{n}\right) + \Theta\left(\frac{2}{n}\right)$$

$$= \frac{1}{2} \times \Theta(n) - \frac{3}{2} \times \Theta(1) + 3 \times \Theta\left(\frac{1}{n}\right) + 2 \times \Theta\left(\frac{1}{n}\right)$$

**Removing all constants:**

$$= \Theta(n) + \Theta\left(\frac{1}{n}\right) + \Theta\left(\frac{1}{n}\right)$$

**When combining two Big Theta  $\Theta$  terms , largest growth rate is the result:**

$\rightarrow \Theta(N)$  grows linearly with `N` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$  decreases to a constant , i. e. when N grows larger ,  $\frac{1}{N}$  approaches to 0.

*This we can prove by limit and continuity also:*

$$\lim_{n \rightarrow \infty} \left( \frac{1}{n} \right) \Rightarrow \frac{1}{\infty} = 0 \left[ \frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

*As  $\Theta(N)$ 's growth rate is higher, therefore, average case is  $\Theta(N)$ .*

---

*Hence, Average Time Complexity is :  $\Theta(N)$ .*

---

*Note, by relation:  $\Omega \leq \Theta \leq O$ , we get:*

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{n^2 - 3n + 6}{2n} + \frac{2}{n} \leq c_2 \times N$$

---

*Average Case Time Complexity[Most Simplest Alternative Approach]:*

*(Simplest approach)*

*For position 1, the probability of deletion is  $\frac{1}{N}$ .*

*For position 2, the probability of deletion is  $\frac{1}{N}$ .*

*.*

*.*

*For position N, the probability of deletion is  $\frac{1}{N}$ .*

*Hence probability of deletion will always remain  $\frac{1}{N}$ , for position 1 to N.*

***Average case complexity : Probability of deletion  $\times$   $\left( \begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$***

$$= \frac{1}{N} \times \left( \sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left( \frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

***Applying Big Theta in the equation:***

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

***Now removing the constant ' $\frac{1}{2}$ ', we get:***

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

***Hence, Average Time Complexity is :  $\Theta\left(\frac{N+1}{2}\right)$  or  $\Theta(N)$ .***

---



*Now,*

*Note , by relation:  $\Omega \leq \Theta \leq O$  , we get:*

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

---

| <i>Multi – level<br/>Linked List</i> | <i>Cases</i>           | <i>Time Complexity</i> |
|--------------------------------------|------------------------|------------------------|
| <i>Delete At<br/>Position</i>        | <i>1. Best Case</i>    | $\Omega(1)$            |
|                                      | <i>2. Worst Case</i>   | $O(n)$                 |
|                                      | <i>3. Average Case</i> | $\Theta(n)$            |

\*\*\*\*\*